

FMC 2026

Novo sistema de controle e WPILib 2027

Sumário

1. A ideia de programação da FRC continua a mesma.....	2
2. Java continua fortemente suportado	2
Para nossos times	2
3. O novo controlador é significativamente mais poderoso	3
4. A interface Web deverá melhorar muito o diagnóstico	3
5. Problemas de CPU e memória devem diminuir bastante	4
7. O código de 2026 NÃO deverá simplesmente compilar em 2027	5
8. Mas o WPILib continuará muito parecido	5
9. Haverá reorganização de métodos, constantes e pacotes	5
10. SmartDashboard e Shuffleboard APIs serão removidas	6
Para nossos times	6
11. O sistema de coordenadas do campo vai mudar.....	7
12. Rotation2d também terá mudança de comportamento	7
13. Maior precisão temporal.....	8
14. WPILib continuará Open Source	8
15. FRC e FTC estão convergindo tecnologicamente	8
Conclusão.....	9

1. A ideia de programação da FRC continua a mesma

Esse talvez seja o ponto mais importante para tranquilizar as equipes.

Apesar da mudança de hardware, a filosofia continua sendo **abstrair os detalhes eletrônicos de baixo nível**. A equipe deve continuar pensando em controlar motores, sensores e subsistemas, sem precisar implementar comunicação CAN, PWM etc.

Portanto, para quem atualmente programa utilizando Java + WPILib:

a lógica geral permanece.

Continuaremos trabalhando com conceitos como:

- subsistemas;
- comandos;
- motores e sensores;
- odometria;
- controle;
- autonomous;
- telemetria.

A grande mudança está na **plataforma e em algumas APIs**, não na filosofia de programação.

2. Java continua fortemente suportado

Foram apresentadas quatro linguagens para o novo ecossistema:

Java, C++, Python e Blockly.

Blockly é uma novidade, inicialmente mais direcionada ao FTC e provavelmente só chegando de maneira mais completa ao FRC posteriormente.

O palestrante reforça que o **Java continuará sendo linguagem de primeira classe**, inclusive mencionando que o FRC atualmente está próximo de uma "monocultura" Java devido à enorme adoção da linguagem pelas equipes.

Para nossos times

Não vejo motivo para migrarmos de Java por causa da mudança de controlador.

Todo o conhecimento que estamos construindo atualmente continua extremamente relevante.

3. O novo controlador é significativamente mais poderoso

O novo System Core apresentado utiliza como base um **Raspberry Pi Compute Module 5**, com processador ARM 64 bits e **4 GB de RAM**.

Entre os recursos apresentados estão:

- 5 barramentos CAN;
- 6 portas de I/O inteligentes;
- 4 USB;
- 2 USB-C de alta velocidade;
- Wi-Fi integrado;
- display integrado;
- LEDs;
- interface Web;
- Linux com suporte de tempo real.

O Wi-Fi integrado não substitui necessariamente o rádio do FRC; o palestrante deixa claro que o rádio separado continuará sendo utilizado no FRC.

4. A interface Web deverá melhorar muito o diagnóstico

Um recurso que me chamou bastante atenção foi a nova **interface Web do controlador**.

Segundo a apresentação, ela mostra muito mais informações que o roboRIO e foi pensada também como uma ferramenta de diagnóstico e debugging.

Isso pode facilitar bastante situações como:

"O motor não está funcionando."

Em vez de começar desmontando código e cabeamento sem informações, teremos potencialmente muito mais dados do sistema disponíveis diretamente pelo controlador.

5. Problemas de CPU e memória devem diminuir bastante

Essa é uma melhoria enorme.

O palestrante cita explicitamente problemas atuais como:

- loop overruns;
- limitações de memória;
- dificuldades de equipes iniciantes para diagnosticar esses problemas.

O System Core, com processador muito mais poderoso e **4 GB de RAM**, foi pensado justamente para diminuir essas limitações.

Para Java existe ainda outra vantagem: a arquitetura 64 bits permitirá utilizar **garbage collectors mais modernos**, com menores tempos de pausa.

Isso é particularmente interessante para equipes que utilizam bastante:

- AdvantageKit;
- visão;
- PathPlanner;
- múltiplas câmeras;
- odometria;
- logging;
- algoritmos mais pesados.

6. Poderemos rodar muito mais coisas diretamente no controlador

A potência adicional abre possibilidades que atualmente seriam difíceis ou pouco recomendadas no roboRIO.

A apresentação menciona:

VS Code/VS Codium rodando no próprio controlador, ferramentas Web, interface de desenvolvimento e até projeto integrado para processamento de visão.

No FRC, entretanto, a expectativa continua sendo que as equipes utilizem principalmente ferramentas de desenvolvimento no computador.

7. O código de 2026 NÃO deverá simplesmente compilar em 2027

Esse é provavelmente o **maior alerta da palestra**.

Foi dito explicitamente que haverá **breaking changes no software em 2027**.

Ou seja:

Não devemos esperar pegar o projeto WPILib 2026, importar para 2027 e simplesmente compilar.

Haverá trabalho de migração.

Isso é muito importante para nossos times saberem antecipadamente.

8. Mas o WPILib continuará muito parecido

Apesar dos breaking changes, o palestrante reforça que no nível superior **muita coisa continuará familiar**.

As linguagens permanecem, o desenvolvimento desktop permanece e muitas das classes continuam semelhantes.

Então não será:

"Precisamos reaprender programação FRC do zero."

Será mais próximo de:

"Precisamos aprender a nova versão da WPILib e migrar nossos conhecimentos."

Isso é uma diferença importante.

9. Haverá reorganização de métodos, constantes e pacotes

Parte da incompatibilidade será resultado de uma grande limpeza realizada no WPILib.

Foram citados:

- reorganização de packages;

- reorganização de namespaces no C++;
- mudanças de nomes de métodos;
- padronização de constantes;
- remoção de determinados padrões antigos.

A justificativa é que o WPILib acumulou aproximadamente 15 anos de desenvolvimento e essa mudança de plataforma criou uma oportunidade para limpar a arquitetura.

Portanto, podemos encontrar coisas como:

"Essa classe ainda existe, mas mudou de lugar."

ou

"Esse método agora possui outro nome."

10. SmartDashboard e Shuffleboard APIs serão removidas

Esse merece bastante destaque para programação.

Segundo a apresentação:

as APIs de SmartDashboard e Shuffleboard serão removidas.

No lugar delas haverá **novas APIs de telemetria e parâmetros ajustáveis**.

A ideia é abandonar coisas como:

```
putNumber()  
getNumber()
```

em favor de uma arquitetura de telemetria mais moderna.

Para nossos times

Isso significa que não é interessante investir excessivamente em arquiteturas novas baseadas diretamente em SmartDashboard pensando em longo prazo.

11. O sistema de coordenadas do campo vai mudar

Esse é outro ponto **muito importante para equipes que trabalham com Swerve, PathPlanner, visão e odometria.**

A origem do sistema de coordenadas será alterada para:

o centro do campo.

Também haverá alteração de orientação em relação ao sistema utilizado em 2026.

O palestrante menciona que isso está sendo coordenado no ecossistema, envolvendo ferramentas como Limelight e AdvantageScope.

Isso vai impactar diretamente:

- Pose2d;
- trajetórias;
- visão;
- autonomous;
- field-relative;
- AprilTags;
- PathPlanner;
- AdvantageScope;
- sistemas próprios de odometria.

Esse é um dos itens que eu colocaria em **vermelho** na nossa documentação de migração 2027.

12. Rotation2d também terá mudança de comportamento

Atualmente havia comportamento que permitia manter um ângulo acumulado.

Na nova implementação, segundo a palestra, a orientação ficará representada dentro de uma volta. Caso a equipe queira acompanhar múltiplas rotações continuamente, precisará manter essa informação separadamente.

Isso merece atenção principalmente em mecanismos rotacionais contínuos.

13. Maior precisão temporal

Internamente, o WPILib passará de **microsegundos** para **nanossegundos** em determinadas representações inteiras de timestamp.

Para a maioria das equipes isso será praticamente transparente, porque muitas APIs trabalham em segundos, mas internamente aumenta a precisão disponível.

14. WPILib continuará Open Source

Essa também foi uma preocupação discutida devido à aproximação entre FRC e FTC.

O palestrante deixa claro que o **WPILib continuará com desenvolvimento aberto**, mesmo com a integração dos ecossistemas.

Isso é ótimo para as equipes avançadas e para a comunidade.

15. FRC e FTC estão convergindo tecnologicamente

Existe uma mudança estratégica maior por trás de tudo isso.

A intenção é aproximar FRC e FTC utilizando uma base comum de hardware/software, principalmente do ponto de vista do controlador.

Isso potencialmente melhora a progressão:

FTC → FRC

porque o aluno pode chegar ao FRC já familiarizado com parte do ecossistema.

Conclusão

Eu resumiria em **6 alertas principais para 2027**:

① **Não abandonar Java.**

Java continuará sendo uma das principais linguagens do FRC.

② **Não precisamos reaprender FRC do zero.**

Command-Based, subsistemas, controle, odometria etc. continuam sendo conhecimentos válidos.

③ **Precisaremos migrar código.**

Projetos de 2026 não devem simplesmente compilar em 2027.

④ **Revisar SmartDashboard/Shuffleboard.**

As APIs atuais serão substituídas por uma nova arquitetura de telemetria.

⑤ **Atenção máxima às coordenadas.**

Swerve, PathPlanner, visão e autonomous precisarão ser revisados por causa do novo sistema de coordenadas.

⑥ **Começar os testes antes da Season 2027.**

Se tivermos acesso antecipado ao novo controlador/ecossistema, vale montar um projeto mínimo com drivetrain, CAN, motor, encoder, odometria, telemetria e autonomous antes do kickoff.

Aliás, essa palestra responde diretamente àquela dúvida que você tinha me feito quando conversávamos sobre a mudança do **roboRIO para o System Core**: **o estilo geral da programação Java permanece, mas 2027 terá mudanças suficientemente grandes no WPILib para exigir adaptação e migração dos projetos.**